

RAG & retrieval: feeding an LLM source-of-truth instead of trusting its memory

How to read this file: the first half is GENERAL RAG/retrieval knowledge (transferable to any system/job/interview — vector search, ANN, chunking, rerankers, hybrid search, MCP). The second half (marked "WORKED EXAMPLE") is how the WCX repo instantiates it on Azure AI Search. Learn the general model; use the repo to make it concrete. **Trust status:** repo specifics verified against live code 2026-07-28: the hybrid `search()` params (search.py:44,54-57: `k_nearest_neighbors=50` , `fields="contentVector"` , `exhaustive=True` , `query_type="semantic"` , `semantic_configuration_name="default"` , `query_caption="extractive"`); the retriever/reranker/caption metadata (search_relevant_docs.py:61-65); `doc_search_index` per-product + `DOC_SEARCH_SERVICE="cpc-search-sre-copilot"` (product.py:47,57,74-149); and the indexing-absence proof. Regrep symbols before citing line numbers. General concepts are stable industry knowledge. **Key boundary (high-trust conclusion): this repo QUERIES only; the data source + indexer + skillset that FILL the index live in Azure config, NOT this repo** — proven by grep-absence (no `Search` / `indexer` / `datasource` / `SplitSkill` / embedding pipeline anywhere in the MCP; the only `datasource` / `ingest` hits are Kusto/Grafana, an unrelated data plane), not by assumption. **Related:** `[[kb-agent-loop]]` (the tool-use protocol that injects retrieved docs and re-calls search), `[[kb-auth-infra]]` (`get_azure_credential` under the search client), `[[kb-distributed-systems-patterns]]` (singleton caching, consumer-of-managed-service boundary), `[[kb-claude-api]]` (tool-use / MCP from the LLM side). `[[kb-harness-engineering]]` (the capstone — retrieval is L2 tools + L3 memory).

PART 0 — WHY THIS MATTERS (philosophy & real-world connection)

Deep principle in one line: don't make the model recall from lossy frozen memory — give it the source-of-truth to read, so answers are current, grounded, and citable.

Why it exists / what it solves. An LLM's knowledge is frozen at its training cutoff and stored *lossily* in weights — it "sort of remembers" the internet, and it has never seen your private/internal data at all. That produces two failures: staleness (can't know last week's incident) and hallucination (with nothing in front of it, a next-token predictor confidently invents). RAG is the dominant production pattern for grounding an LLM in your own/current data *without retraining*: you retrieve the relevant source and let the model read it. It beats **fine-tuning** for knowledge because you update an index in seconds instead of re-training, and answers stay citable/auditable (fine-tuning is for behavior/format, not fresh facts). It beats **just-paste-it-all-in long-context** because it scales — you retrieve the relevant slice instead of paying tokens for (and drowning recall in) the whole corpus.

Real-world connection — where you actually meet it. Nearly every enterprise "chat with your docs" product, customer-support bot, code assistant grounding on a repo, and search-and-answer engine (Perplexity) is RAG under the hood; internal knowledge bots too. The same embedding/vector-search machinery independently powers semantic search, recommendations, dedup, and clustering — and vector DBs (Pinecone, pgvector, Qdrant, Weaviate, Azure AI Search) are a whole market segment. Building RAG well is one of the most in-demand LLM-engineering skills in 2026.

What breaks without understanding it:

- **Silent context-loss hallucination** — if the answer sits in a chunk you never retrieved, the model won't reliably backfill it; it fabricates. Chunks are not a safety net.
- **Bad chunking → garbage retrieval** — cut mid-idea and every downstream stage inherits the mess; chunking is an indexing-time decision that dominates query quality.
- **Vector-only search misses exact IDs/error codes** — "KB5034441" must match *exactly*, not "sort of like a KB article" — which is the whole reason **hybrid** (keyword + vector + rerank) exists.
- **Brute-force at scale is too slow** — O(N) per query is fine for thousands, deadly for millions; that's why **ANN** indexes exist.
- **Confusing indexing vs query phase** — "why can't it find my new doc?" almost always means it was never *ingested* into the index, not that the query is wrong.

Transferable mental model. Retrieval = turn *meaning into geometry*, find what's nearby, rerank the shortlist, inject it into the context window. Hold two phases apart: building the index (offline) vs serving a query (online). And know your seat: in practice you almost always *consume* a search service and build only the query side — the index-filling pipeline is usually someone else's (here, Azure's).

PART 1 — GENERAL KNOWLEDGE

1.1 What RAG is and why it exists

RAG = Retrieval-Augmented Generation. Before the LLM answers, you *retrieve* relevant source documents and *augment* the prompt with them, so the model reads facts instead of recalling them. It exists to fix two structural failures of a bare LLM:

- **Frozen, fuzzy memory.** The model's knowledge is baked in at training time (a cutoff date) and stored lossily in weights — it "sort of remembers" rather than holding an exact copy. It can't know your internal TSGs, last week's incident, or private docs at all.
- **Hallucination.** With no source in front of it, a model will fluently invent plausible-but-wrong answers, because it's a next-token predictor, not a database.

RAG's fix: **read, don't recall.** Feed the source-of-truth into the context window → answers become accurate for *your* data, current, and **citable** (you can point at the passage it used). (Damien derived this: raw LLM = frozen/fuzzy + hallucinates → inject real docs → accurate + citable.)

The **R-A-G breakdown**:

- **R (Retrieve):** search a knowledge store for chunks relevant to the question (the only part you usually build — an embedding/keyword search).
- **A (Augment):** stitch those chunks into the prompt/context (agent-loop plumbing — a `tool_result` injection, not ML).
- **G (Generate):** the LLM writes the answer grounded in the injected chunks.

RAG vs fine-tuning vs long-context (the "which knob" question):

- **RAG** — knowledge that changes often, is large, or must be citable/auditable. Update the index, not the model. Best default for "answer over my docs."
- **Fine-tuning** — teach *behavior/format/style/tone* or a narrow skill, not fresh facts. Expensive, static, hard to cite, goes stale. Don't fine-tune to add knowledge you could retrieve.
- **Long-context (just paste it all in)** — when the corpus is small enough to fit and you want zero retrieval infra. Costs tokens per call, has a ceiling, and "lost in the middle" recall degrades. RAG is long-context's scalable cousin: retrieve the *relevant* slice instead of pasting everything.

1.2 Embeddings & vector search (the semantic engine)

An **embedding** is a fixed-length vector of floats (e.g. 768 / 1536 / 3072 dims) that an embedding model produces from text. The trick: **semantically similar text lands geometrically close** in that high-dimensional space. "reset the VM" and "restart the virtual machine" point in nearly the same direction even with zero shared words — that's what keyword search can't do.

Similarity metrics (how you measure "close"):

- **Cosine similarity** — the angle between two vectors, ignores magnitude, range $-1..1$. The default for text; robust to differences in vector length. (WCX effectively rides on this.)
- **Dot product** — cosine *scaled by* magnitudes. Equivalent to cosine when vectors are normalized (many models pre-normalize, so dot == cosine). Cheaper to compute.
- **Euclidean (L2) distance** — straight-line distance; sensitive to magnitude. Less common for text embeddings.

Dimensionality trade-off: more dims = more expressive but more storage + slower search + more data needed. Some models support *shortening* (Matryoshka) to trade a little accuracy for speed.

The scaling problem — ANN vs brute force. Comparing a query vector to *every* stored vector (exhaustive / brute-force / "flat") is exact but $O(N)$ per query — fine for thousands, deadly for millions. **Approximate Nearest Neighbor (ANN)** indexes trade a sliver of recall for massive speed:

- **HNSW** (Hierarchical Navigable Small World) — a layered proximity graph you greedily walk; low latency, high recall, more memory. The most common default (Azure AI Search, pgvector, Qdrant, Weaviate all offer it).
- **IVF** (Inverted File) — cluster vectors into cells, search only the few nearest cells. Often paired with **PQ** (Product Quantization) to compress vectors. Great memory footprint at scale.
- **Exhaustive/flat** — no index; exact but slow. Still used for small sets or when you set `exhaustive=True` to force exactness (WCX does exactly this — see §2.4).

Vector databases store vectors + metadata and serve ANN queries: **Pinecone** (managed SaaS), **Weaviate**, **Qdrant** (open-source, self- or cloud-host), **pgvector** (a Postgres extension — vectors live next to relational data, no new system), **Azure AI Search** (Microsoft's

managed search that does vector *and* keyword *and* semantic rerank in one service — what WCX uses). Trade-offs: managed vs self-hosted ops burden; dedicated vector store vs bolt-on (pgvector) vs full hybrid engine (Azure); cost, scale ceiling, and whether you also need keyword/filtering in the same query.

1.3 The two-phase pipeline (the split people conflate)

RAG is **two separate pipelines** that share one index. Confusing them causes most "how does it even know what to search" confusion.

INDEXING (build the index — offline, ahead of time): ingest docs → chunk → embed each chunk → store (vector + text + metadata) . Runs on a schedule/trigger, independent of any user question.

QUERY (serve a question — online, per request): embed the query → retrieve nearest chunks → (rerank) → return . This is the only phase WCX's code touches.

Chunks, not documents (fundamental). You don't embed whole documents — you split them into **chunks** and embed/rank/return *chunks*. Why: a single vector can only summarize so much; a 40-page doc squished to one vector is mush, and you want to return the *paragraph* that answers, not the whole PDF. So two chunks from the same doc score independently (no aggregation), and "50 results" means 50 chunks (from possibly fewer docs) — they were never "50 docs then chopped."

Chunking strategies (an INDEXING-time decision with big query-quality consequences):

- **Fixed-size** (N tokens/chars) — simple, but blindly cuts mid-sentence/mid-idea.
- **Overlap** (each chunk repeats the last ~10-20% of the previous) — so a fact straddling a boundary survives in at least one chunk. Cheap insurance against the boundary-loss failure.
- **Semantic / recursive** — split on structure (headings → paragraphs → sentences) so chunks align with ideas, not arbitrary offsets.
- **Parent-document retrieval** — embed small precise child chunks for *matching*, but return the larger parent chunk for *context*. Best of both: precise retrieval, complete context.

1.4 Retrieval quality: retriever vs reranker (the 2-stage funnel)

The core move in production retrieval is **cheap-and-wide, then expensive-and-sharp**:

- **Stage 1 — retriever (bi-encoder / "two-tower").** Query and chunk are embedded **separately** (the chunk's vector was precomputed at indexing; only the query is embedded at query time), then compared by vector distance. Fast, scales to millions, but **blunt**: squishing each side to one vector independently loses cross-token nuance — it can miss negation ("restart" vs "do NOT restart") because both vectors look similar.
- **Stage 2 — reranker (cross-encoder).** Take the top ~50 from stage 1 and feed query + chunk **together** into one model, with attention flowing across both, producing one relevance score per pair. **Slow, sharp** — catches nuance and negation. Too expensive to run over millions (that's why stage 1 pre-filters), just right over 50.

The funnel: retriever is too blunt for final order; reranker is too costly for the whole corpus — so chain them. Retriever nominates candidates; reranker orders them.

Hybrid search — one question, three lenses in ONE call. Modern engines run three mechanisms in parallel on the *same* raw query string and fuse the results:

- **Keyword / BM25** — literal term match. Still essential: exact IDs, error codes, product names, GUIDs, rare tokens — things semantic vectors blur ("KB5034441" must match *exactly*, not "sort of like a KB article").
- **Vector** — semantic nearest-neighbor (the \$1.2 engine).
- **Semantic rerank** — the cross-encoder pass (\$1.4 stage 2) reorders the fused candidates. Vector alone fails on exact identifiers; keyword alone fails on paraphrase — hybrid covers both, then reranks.

The context-loss failure mode of chunk-RAG (its central weakness). If the answer depends on a detail that sits in a chunk you *didn't* retrieve, the LLM does **not** reliably backfill it — it may silently hallucinate. Chunks are not a safety net. Real fixes live in **retrieval design**: chunk overlap, parent-document retrieval, return more chunks, semantic-boundary chunking. The only legit runtime backfill is the **agent loop** re-calling search with a refined query *if it notices the gap* (see [[kb-agent-loop]]). Chunk-RAG deliberately trades context-completeness for precision + token-efficiency.

1.5 MCP (Model Context Protocol) — the tool-exposure standard

MCP is an open, industry-standard protocol (originated by Anthropic, now broadly adopted) for exposing tools, prompts, and resources to an LLM over a uniform interface. Think "USB-C for LLM context": instead of every app hand-wiring its own function-calling glue, an MCP **server** advertises capabilities and any MCP **client** (a host app / agent runtime) can discover and call them.

- **Relation to function-calling:** function-calling is the LLM-level primitive — the model emits a structured "call tool X with these args," you run it, you feed back a `tool_result` (see `[[kb-claude-api]]`). MCP is the **transport + packaging standard** around that: it standardizes *how tools are described, discovered, and invoked across process/network boundaries*, so the same tool server works with any compliant client. Function-calling is the conversation; MCP is the plug.
- **Server/client model:** the **server** hosts the tools (does the work); the **client/host** owns the LLM and the loop (decides *which* tool and *when*); they talk over a defined channel (HTTP, stdio). This separation means you can swap the model without touching tools, and reuse tools across agents.
- **Three exposable kinds:** **tools** (functions the model CALLS mid-loop), **prompts** (pre-written templates that START/shape a session), **resources** (readable data the client can pull in).

1.6 Supporting patterns (general, then repo-instantiated in Part 2)

- **Wrapper / adapter pattern.** Put your own thin class in front of a fat third-party SDK: a smaller interface (4 params vs ~10), encapsulated setup, uniform logging, and **swappability** — the wrapper is the single seam between your code and the vendor. (Adapter = same idea specifically to make a foreign interface match yours.)
- **Client caching / lazy singleton.** Constructing a service client can be expensive (auth handshake, connection setup). So build it **once, on first use**, cache it keyed by its config, and hand the cached instance to everyone. **Double-checked locking** makes that thread-safe without paying a lock on every access. The key insight: cache the *expensive-to-build* thing, keyed by what makes it distinct.
- **Separation of concerns / layering.** Each layer has one job and knows only the layer below it: tool (assemble request) → resource (hand a cached client) → client class (do the HTTP) → managed service. Swap the transport by editing only the client layer; everything above is untouched. See `[[kb-distributed-systems-patterns]]`.

PART 2 — WORKED EXAMPLE: the WCX Azure AI Search query pipeline

WCX is a textbook instance of §1.3's **query-only** half of RAG (someone else built the index) bridged through an MCP tool (§1.5) into Azure AI Search's **hybrid** engine (§1.4). "RAG = a PATTERN, not a technology" (front-wheel drive, not the engine): **R** = `search_relevant_docs.py` → Azure AI Search (the only code written); **A** = `tool_result` injection (agent-loop plumbing); **G** = the LLM reply.

2.1 MCP server architecture — 3 layers by job (§1.5 made concrete)

- **SERVER** = `diagnostics-mcp.py` (~55 lines): `FastMCP("diagnostics-mcp", stateless_http=True)` + `register_tools(mcp)` + `register_prompts(mcp)`. `FastMCP` implements the `tool_use/tool_result` protocol so you register plain Python fns.
- **REGISTRAR** = `core/tools/__init__.py`: auto-discovery via `pkgutil.iter_modules` (drop a `.py` file with a public fn = registered, no manual list). Guards: `__module__` check, `_should_register()` gate, uniform-logging decorator.
- **TOOLS** = individual fns. **Signature + docstring + Pydantic Field(description=)** **ARE the schema the LLM sees** — the tool contract is the code. Pydantic = "type hints that do something" — runtime validation + auto JSON schema; `ge=1, le=20` on `max_records` doubles as a context-window guardrail (`search_relevant_docs.py:31-34`).

MCP servers expose BOTH tools AND prompts (`register_tools` + `register_prompts` , §1.5's three kinds). Tool = fn the LLM CALLS during the loop; prompt = pre-written template that STARTS/shapes the session (`troubleshoot_incident` is a PROMPT-builder, not a tool).

Foundry ↔ MCP contract (§1.5 server/client): Foundry/agent owns LLM + loop (which tool + when); MCP owns tools (does the work); they talk over HTTP. Mechanical test for "which side is this file": under `src/mcp-servers/` ? → MCP side. Statelessness recurs at every layer (`stateless_http=True`) — push state to ONE place (the thread), keep the rest stateless.

2.2 The wrapper — TWO classes named `SearchClient` (§1.6 wrapper)

In `utils/search.py`: Azure's SDK class is imported aliased — `from azure.search.documents import SearchClient as AzureSearchClient` (the `as` alias is the wrapper tell) — and the repo defines its **own** `SearchClient` that wraps it. The wrapper's `search()` takes 4 params (`query`, `search_type`, `select`, `top`) and hides the ~10-param Azure call + vector-query setup +

timing/logging. Ownership ladder: tool → our `SearchClient` wrapper → `AzureSearchClient` SDK → the rented Azure AI Search service. The wrapper is the seam between our code and Azure's.

2.3 Layering + the cached client (§1.6 lazy singleton)

Tool assembles the request → `utils/resource.py`'s `get_search_client(...)` hands back a **cached** client → the `SearchClient` class does the real HTTP (3 hops). Swap transport by editing only the client. The **ResourceManager singleton** caches expensive clients keyed by the config tuple `(service_name, index_name)`, using double-check locking — because constructing a client does an **auth handshake** (`get_azure_credential`, see [\[\[kb-auth-infra\]\]](#)) that's expensive to redo per call.

2.4 The hybrid query — one string, two retrieval lenses fused, then reranked (§1.4, made concrete)

`search_relevant_docs` calls the wrapper with `search_type=SEMANTIC_VECTOR_HYBRID`, `select=["content","title"]`, `top=max_records` (`search_relevant_docs.py:50-52`). The user's question is **NOT rewritten** into different params — the SAME raw string drives everything in one `self._search_clients.search(...)` call (`search.py:49-58`). Note the pipeline order: **two retrieval lenses run in parallel and their results are fused, THEN a reranker reorders the fused shortlist** (rerank is a post-fusion step, not a third parallel retriever — §1.4):

- `search_text=query` → **KEYWORD / BM25** retriever (literal terms — exact error codes / IDs). ↴ run in parallel,
- `vector_queries=[VectorizableTextQuery(text=query, k_nearest_neighbors=50, fields="contentVector", exhaustive=True)]` → **VECTOR** retriever (`search.py:43-45`). ↴ results fused. The only transform is embedding, done *inside* Azure; it compares the embedded query against the precomputed `contentVector` field; `k_nearest_neighbors=50` = pull the top-50 nearest chunk vectors; **exhaustive=True forces brute-force/flat over ANN** (§1.2 — exact, no approximation, viable because this corpus is small).
- `query_type="semantic" + semantic_configuration_name="default"` → **semantic RERANK** applied *after* fusion (§1.4 stage-2 cross-encoder reorders the fused candidates — the sharp pass, not a retrieval lens).
- Plus `select=["content","title"]` (fields to return), `top=max_records` (`≤20` final cutoff), `query_caption="extractive"`, `include_total_count=True`. These params come from the Azure AI Search SDK contract, not this repo.

2.5 Retriever / reranker / caption scores in the result (§1.4 metadata)

Each result carries three Azure-populated fields, read in `search_relevant_docs.py:61-65` :

- `@search.score` → **stage-1 retriever** score (blunt vector/keyword fusion).
- `@search.reranker_score` → **stage-2 reranker** score (sharp cross-encoder, 0-4 range).
- `@search.captions` → a stage-2 **by-product** (`query_caption="extractive"` = the best verbatim sentence(s) with `.text` + `.highlights`) — finer-grained than a chunk. Granularity ladder: document (`title`) → chunk (`content`) → caption (sentences). The 50 candidates ARE chunks (§1.3), narrowed by rerank to `≤ max_records` .

2.6 Chunks + per-product index (§1.3 chunks)

This is a **chunk-level** design (proof: it returns `content` passages + a `title` label, no doc aggregation). Each product routes to its own index via `doc_search_index` on the product config (`product.py:47,74-149`): `engcopilot-w365` , `engcopilot-cdp` , `engcopilot-avd` , `engcopilot-mr` , `engcopilot-w365link` — all on the one service `DOC_SEARCH_SERVICE="cpc-search-sre-copilot"` (`product.py:57`). The tool's `product` arg (`W365/CDP/AVD/MR/W365Link`) selects which index to query — a POINTER, chosen at query time.

2.7 INDEXING vs QUERY — this repo only QUERIES (the boundary, §1.3)

RAG has two phases; **this repo does ONLY the query phase**. Evidence (absence-as-proof, re-verified 2026-07-28): grepping the diagnostics MCP for Search `indexer` / `datasource` / `SplitSkill` / `chunk` / `embedding` -pipeline symbols returns **nothing about Azure Search index-building** — the only `datasource` / `ingest` hits are **Kusto/Grafana** (an unrelated telemetry data plane). The query reads a **pre-computed** `contentVector` field; the repo's skill doc scopes Search to querying; the IaC present deploys SRE agents, not Search indexes.

- **What this repo configures:** WHICH index (`doc_search_index` per product) + WHICH service (`DOC_SEARCH_SERVICE` → `https://{service}.search.windows.net/` , `search.py:25`). A **pointer only**.

- **What Azure configures (outside this repo):** a **data source** (connects to CMD-Docs / eng.ms / TSGs), an **indexer** (scheduled crawler pulling from the data source), a **skillset** (the chunk + embed pipeline that fills `contentVector`). "Where does it search out repos/docs" = the data source; "how does it know to go get them" = the indexer. Set via Portal/CLI/separate pipeline, NOT version-controlled here.
- "How does it know it's *capable* of searching the docs?" — it has no capability list; capability = "those docs were previously ingested into this index." Un-ingested = unsearchable.

Recurring boundary theme: this repo is consistently the **CONSUMER** of a pre-built managed-service artifact (the index / the thread / the agent instructions); the CONSTRUCTION lives across the service boundary. Recognizing that boundary is most of understanding the system. See [\[\[kb-distributed-systems-patterns\]\]](#).

PART 3 — transferable takeaways (the interview/next-job version)

1. **RAG = read, don't recall.** Retrieve source-of-truth into context to beat frozen memory + hallucination, and to make answers citable. Reach for RAG (not fine-tuning) to add *knowledge*; fine-tune only for *behavior/format*.
2. **Embeddings turn meaning into geometry.** Similar text → nearby vectors; compare by cosine (angle, magnitude-blind). This is what keyword search can't do — and what fails on exact IDs, which is why you still need keyword.
3. **At scale, retrieval is approximate (ANN: HNSW/IVF), not brute-force** — you trade a sliver of recall for orders-of-magnitude speed. Force `exhaustive` only when the corpus is small enough.
4. **RAG is two pipelines sharing one index: INDEXING (chunk→embed→store) vs QUERY (embed→retrieve→rerank).** Know which one your code is in — WCX is query-only.
5. **You retrieve CHUNKS, not documents.** Chunking strategy (overlap, semantic, parent-document) is an indexing-time decision that dominates query quality.
6. **Production retrieval is a funnel: cheap-wide retriever (bi-encoder) → expensive-sharp reranker (cross-encoder),** with hybrid keyword+vector+semantic fusion on the same query string.
7. **Chunk-RAG's failure mode is silent context loss** — if the answer's in an unretrieved chunk, the model may hallucinate. Fix it in retrieval design, not by hoping.
8. **MCP is the standard plug for exposing tools/prompts/resources to any LLM client** — server does the work, client owns the loop; it standardizes function-calling across boundaries.
9. **Wrap fat SDKs, cache expensive clients (lazy singleton, double-checked locking), and layer by job** — so you swap a vendor or transport by editing one seam.
10. **This repo is a CONSUMER of managed-service artifacts** (index / thread / instructions); the construction lives across the service boundary — spotting that line is most of the architecture.